



MULTIMEDIA-PROGRAMAZIOA ETA GAILU MUGIKORRAK

MVC (MODELO-VISTA-CONTROLADOR)

ARIKETA: "LAYOUT"

Praktika sorta honen helburua **Android-eko Layout desberdinen erabilera** ikustea da baina **MVC (Model-View-Controller)** egitura mantenduz. Horrela, ez dugu soilik interfazea sortuko, baizik eta logika egoki banatuta izango dugu.

PRAKTIKA01 — LINEARLAYOUT(BERTIKALA)

Aplikazio bat sortuko dugu non erabiltzaileak bi zenbaki sartzen dituen eta batuketa egiten den botoia sakatzean. Emaiza **TextView** batean bistaratuko da. Interfazea **LinearLayout bertikal** batean antolatuta egongo da.

Egituraren eskakizunak

Sortu proiektua Android Studio-n izen **honekin: mvc_linearlayout(V)**

Eta ondorengo pakete-egitura erabili:

```
com.example.mvc_linearlayout(V)
```

```
|— model
```

```
|   |— CalculatorModel.kt
```

```
|— controller
```

```
|   |— CalculatorController.kt
```

```
|— view
```

```
    |— MainActivity.kt
```

Funtzionamendua

1. Erabiltzaileak bi zenbaki sartzen ditu.
2. “**Batu**” botoia sakatzean, **View**-ak **Controller**-era bidaltzen ditu datuak.
3. **Controller**-ak **Model**-ari deitzen dio kalkulua egiteko.
4. **Model**-ak emaitza bueltatzen du.
5. **Controller**-ak **View** eguneratzen du, emaitza erakutsiz.

PRAKTIKA02 – LINEARLAYOUT(HORIZONTALA)

Erabiltzaileak botoi batzuk ikusiko ditu **lerro horizontal batean (LinearLayout)**, eta **botoi** bat sakatzean, testu bat eguneratuko da azken botoiaren balioarekin. Gainera, “**Garbitu**” **botoia** erabil daiteke testua ezabatzeko.

Egituraren eskakizunak

Sortu proiektua Android Studio-n izen **honekin: mvc_linearlayouth**

Eta ondorengo pakete-egitura erabili:

com.example.mvc_linearlayouth

└─ model

| └─ ButtonModel.kt

└─ controller

| └─ ButtonController.kt

└─ view

└─ MainActivity.kt

Funtzionamendua

1. **Model** gordetzen du azken botoiaren balioa.
2. **Controller** kudeatzen du logika: botoia sakatzean **Model** eguneratu eta **View**-ri mezua bidali.
3. **View (MainActivity)** emaitza soilik erakusten du eta **Controller**-era bidaltzen ditu ekintzak.

PRAKTIKA03 – TABLELAYOUT

Botoi-taula bat sortu beharko da (**TableLayout**) non botoi bakoitzak zenbaki bat erakusten duen. **Botoi** bati klik egitean, **Controller**-ak mezua bidaltzen du **Toast** bidez.

Egituraren eskakizunak

Sortu proiektua Android Studio-n izen honekin: **mvc_tablelayout**

Eta ondorengo pakete-egitura erabili:

com.example.mvc_tablelayout

└─ model

| └─ ButtonModel.kt

└─ controller

| └─ ButtonController.kt

└─ view

└─ MainActivity.kt

Funtzionamendua

1. **Model** gordetzen du azken botoiaren balioa.
2. **View**-ak botoiak erakusten ditu (XML-n definituta).
3. **Controller** kudeatzen du logika: botoia sakatzean **Model** eguneratu eta **View**-ri mezua bidali.
4. **View (MainActivity)** emaitza soilik erakusten du eta **Controller**-era bidaltzen ditu ekintzak.

PRAKTIKA04 – FRAMELAYOUT

FrameLayout erabiliz irudi bat ezkutatu eta erakustea **botoi** baten bidez.

Egituraren eskakizunak

Sortu proiektua Android Studio-n izen **honekin: mvc_framelayout**

Eta ondorengo pakete-egitura erabili:

com.example.mvc_framelayout

```
|— model
|   |— ImageModel.kt
|— controller
|   |— ImageController.kt
|— view
|   |— MainActivity.kt
```

Funtzionamendua

1. Hasieran **irudia** ezkutatuta dago.
2. “**Erakutsi irudia**” **botoia** sakatzean, **Controller**-ak **Model**-ari egoera aldatzen dio.
3. **Model**-ak **isVisible** egoera aldatzen du.
4. **Controller**-ak **View**-ri esaten dio irudia erakutsi edo ezkutatzeko.

PRAKTIKA05 – SCROLLVIEW + LINEARLAYOUT

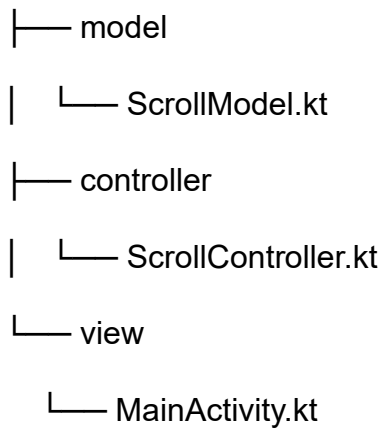
ScrollView baten barruan hainbat **botoi** erakustziko dira. **Botoi** bakoitzak bere testua erakusten duen **Toast** bat bistaratuko du sakatzean. Gainera, azpian dagoen **TextView** batean erakutsiko da azken botoi sakatua.

Egituraren eskakizunak

Sortu proiektua Android Studio-n izen **honekin: mvc_scrollview**

Eta ondorengo pakete-egitura erabili:

com.example.mvc_scrollview



Funtzionamendua

1. **Model** gordetzen du botoien izenak eta azken botoi sakatua.
2. **View** (MainActivity) interfazea erakusten du: ScrollView + LinearLayout + botoiak + TextView.
3. **Controller** botoien logika kudeatzen du eta Model eta View arteko komunikazioa ahalbidetzen du.